

UNIVERSIDAD AUTÓNOMA DE
CHIHUAHUA



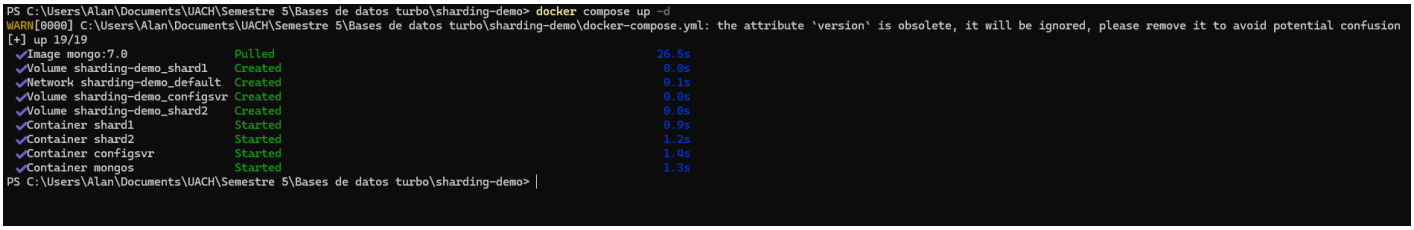
Actividad 1: MongoDB Cluster

Universidad Autónoma de Chihuahua
Campus 2, Facultad de Ingeniería
Bases de datos avanzadas
Alumnos: Alan Fabricio Vega Gutiérrez, Matricula 385658.
Edgar Klassen Saenz, Matricula 385429.

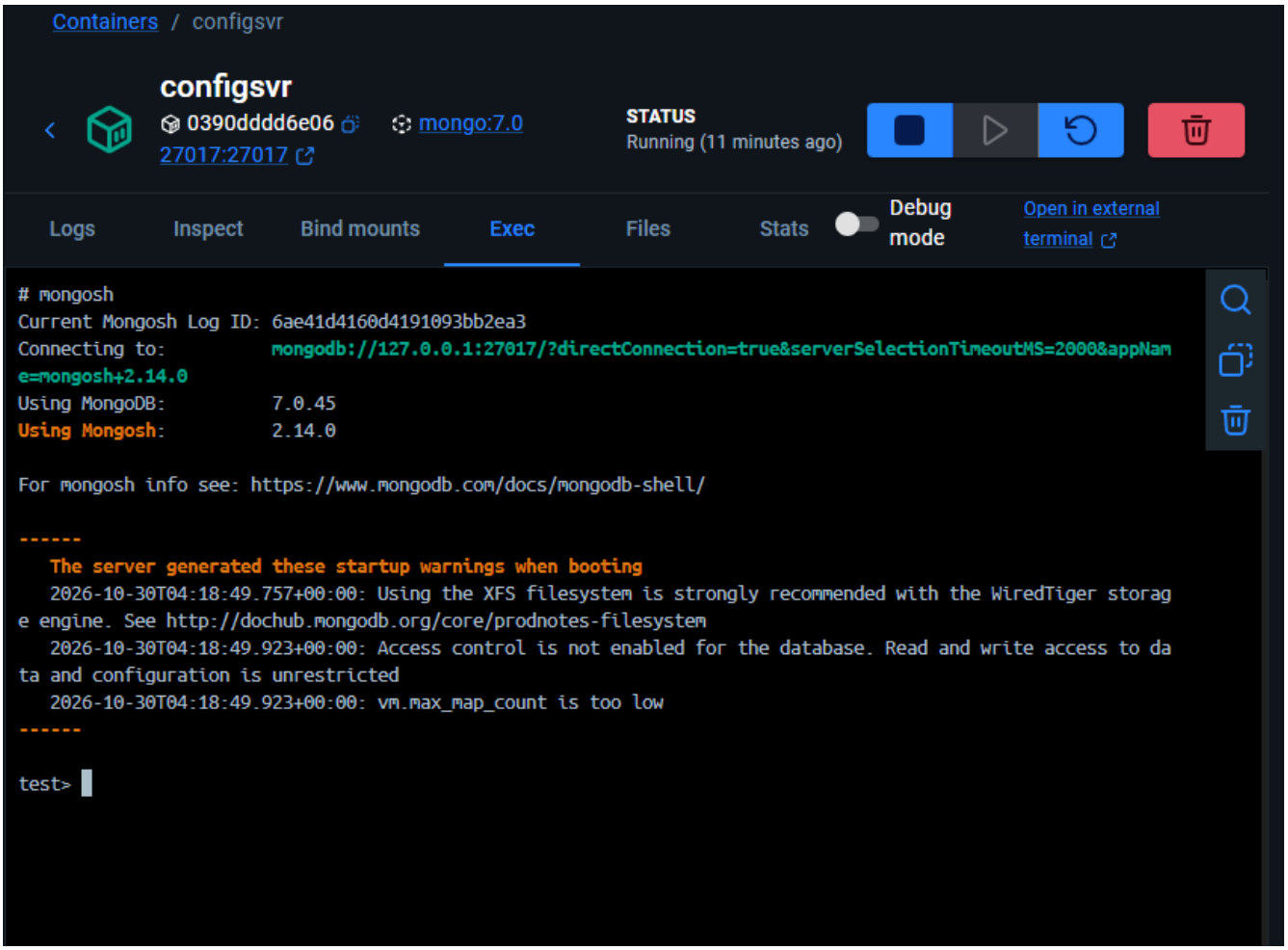
Maestro: Jose Saul De Lira Miramontes

29/10/2026

Iniciamos la imagen en Docker con su sharding.

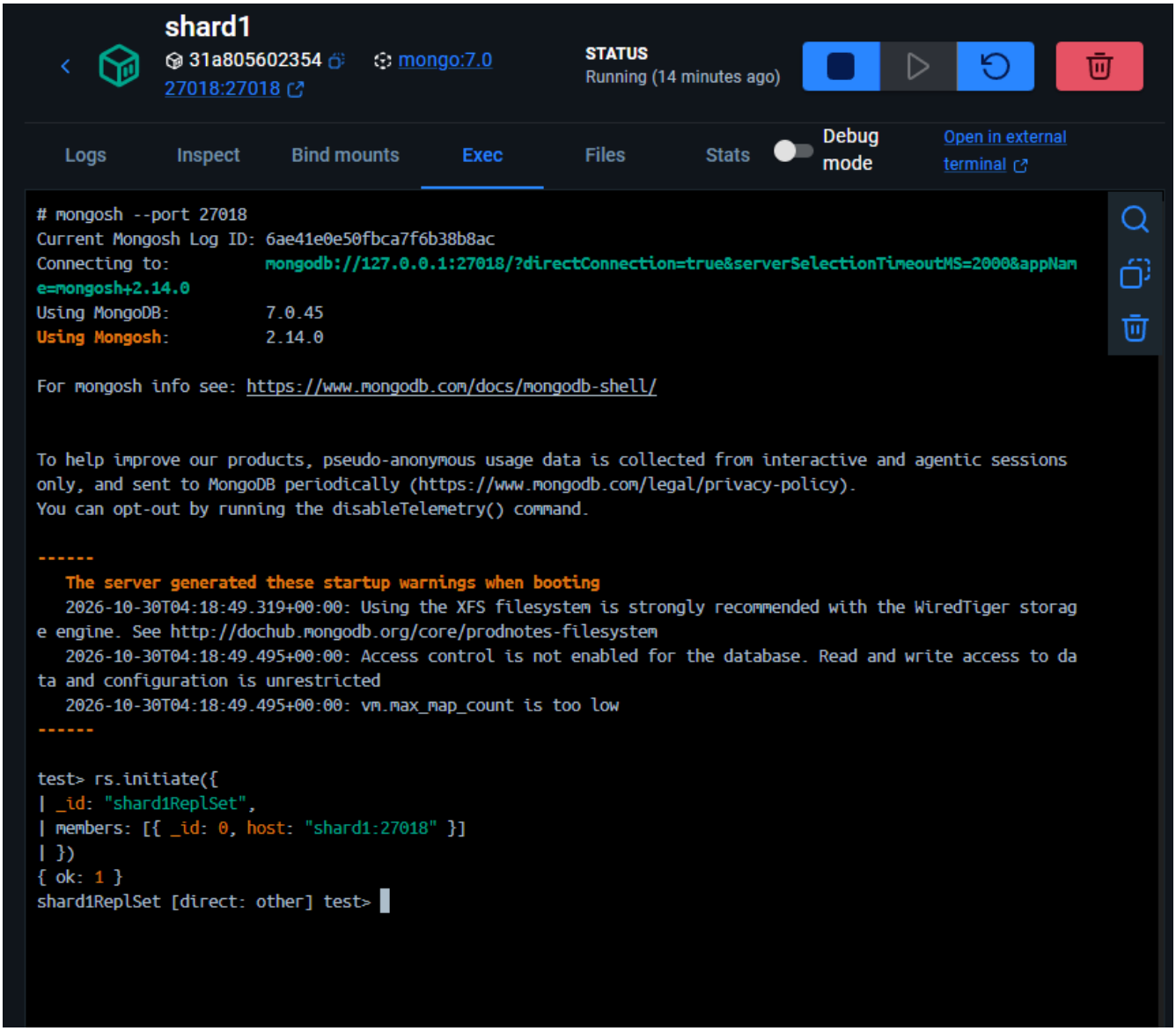


Entramos en la terminal de mongosh para iniciar los replica sets.

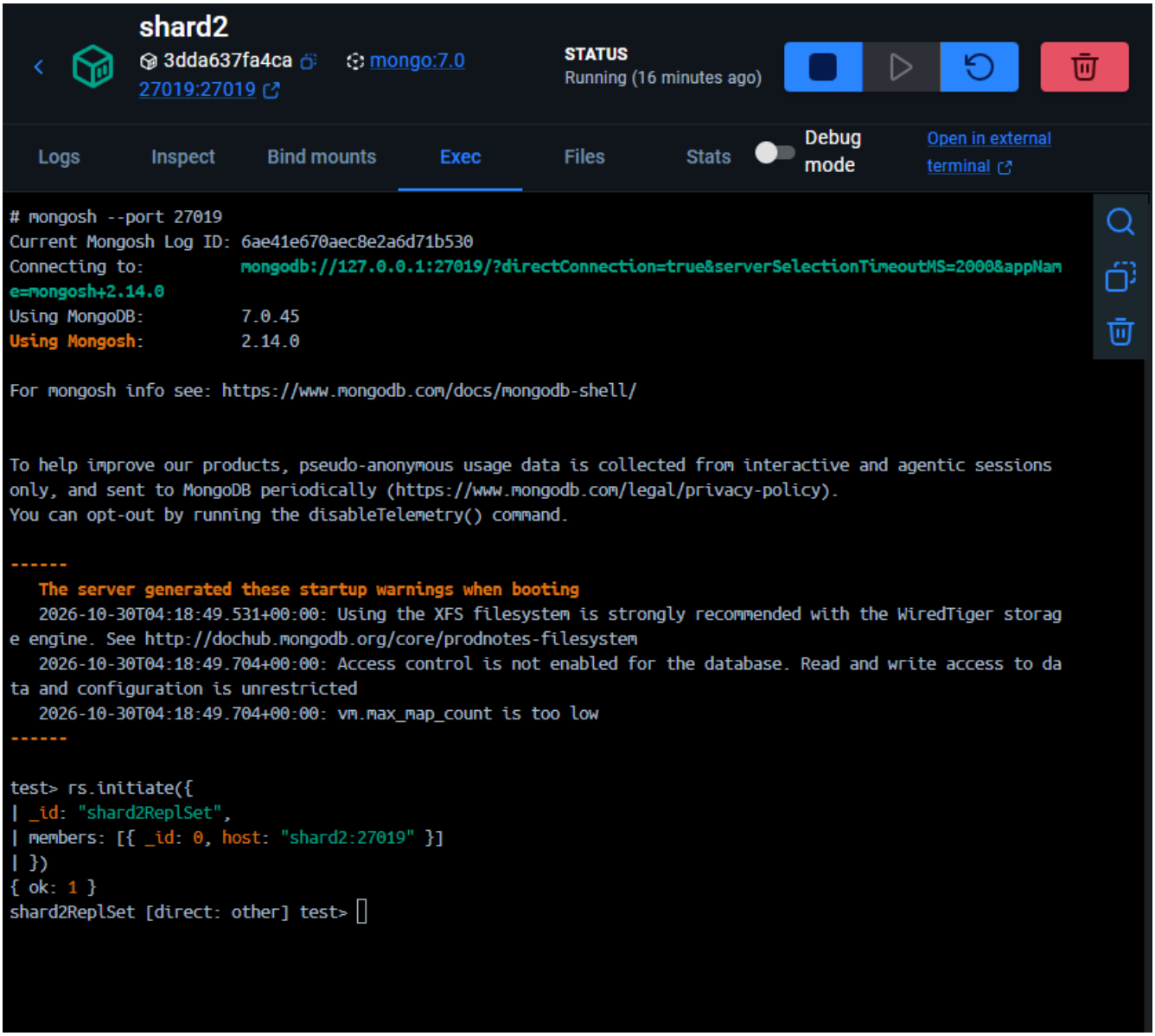




Iniciamos el shard 1



Iniciamos el shard 2



Creando el tercer shard

```yaml
shard1:
  image: mongo:7.0
  command: --shardsvr --replSet shard1ReplSet --port 27018
  container_name: shard1
  ports:
    - 27018:27018
  volumes:
    - shard1:/data/db

shard2:
  image: mongo:7.0
  command: --shardsvr --replSet shard2ReplSet --port 27019
  container_name: shard2
  ports:
    - 27019:27019
  volumes:
    - shard2:/data/db

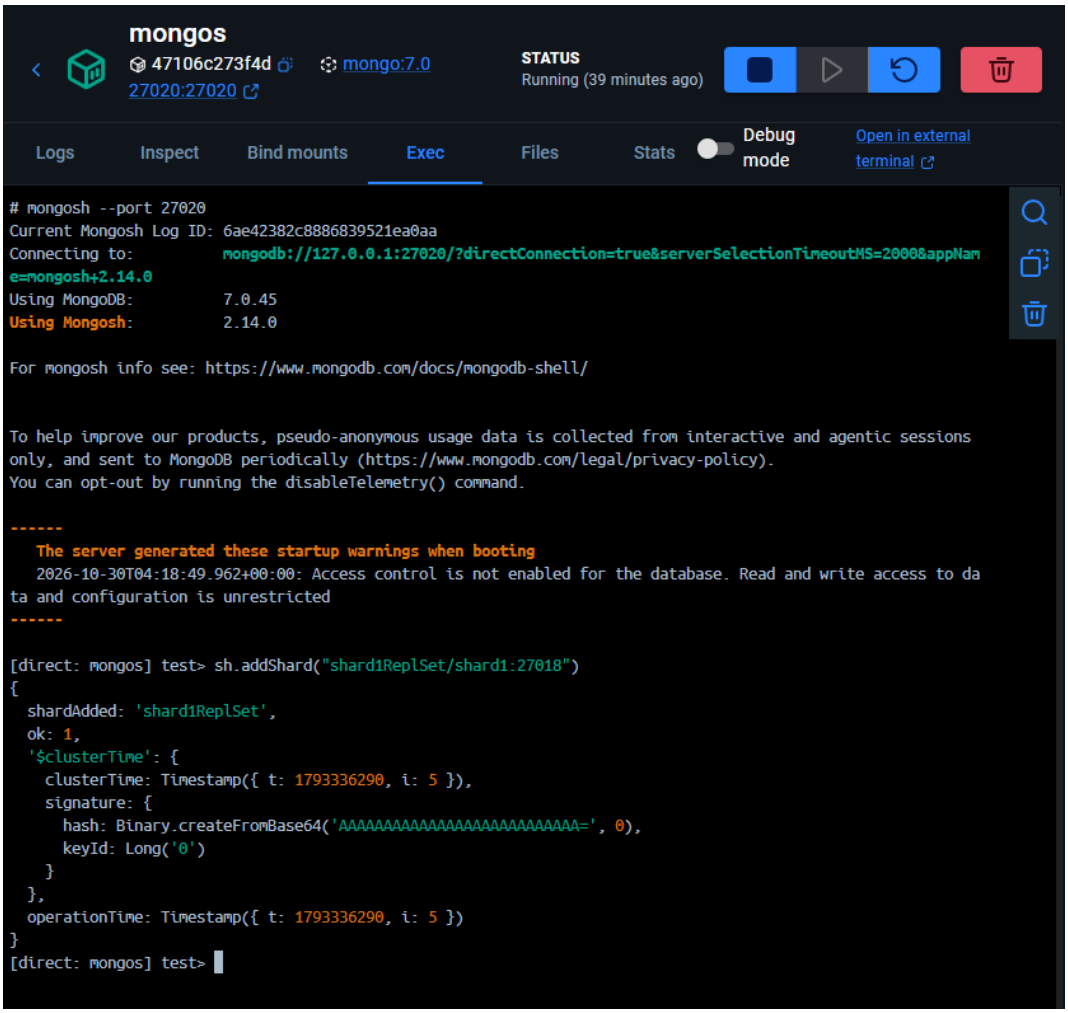
shard3:
  image: mongo:7.0
  command: --shardsvr --replSet shard2ReplSet --port 27019
  container_name: shard2
  ports:
    - 27019:27019
  volumes:
    - shard3:/data/db
```

```
WARN[0000] C:\Users\Alan\Documents\UACH\Semestre 5\Bases de datos turbo\sharding-demo\docker-compose.y
ml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confu
sion
[+] up 1/1
 ✔Container shard3 Started                                                                    0.4s
PS C:\Users\Alan\Documents\UACH\Semestre 5\Bases de datos turbo\sharding-demo>
```



Name	Container ID	Image	Port(s)	Actions
sharding-demo	-	-	-	
configsvr	0390dddd6e06	mongo:7.0	27017:27017	
shard1	31a805602354	mongo:7.0	27018:27018	
shard2	3dda637fa4ca	mongo:7.0	27019:27019	
mongos	47106c273f4d	mongo:7.0	27020:27020	
shard3	3dc1a04e3241	mongo:7.0	27021:27021	

Ahora entramos al router de mongos y añadimos los shards al cluster




```
[direct: mongos] test> sh.addShard("shard2ReplSet/shard2:27019")
{
  shardAdded: 'shard2ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793336342, i: 14 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793336342, i: 4 })
}
[direct: mongos] test>
```

```
[direct: mongos] test> sh.addShard("shard3ReplSet/shard3:27021")
{
  shardAdded: 'shard3ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793336399, i: 4 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793336399, i: 4 })
}
[direct: mongos] test>
```

```
{
  shardAdded: 'shard1ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793336290, i: 5 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793336290, i: 5 })
}
[direct: mongos] test> sh.addShard("shard2ReplSet/shard2:27019")
{
  shardAdded: 'shard2ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793336342, i: 14 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793336342, i: 4 })
}
[direct: mongos] test> sh.addShard("shard3ReplSet/shard3:27021")
{
  shardAdded: 'shard3ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793336399, i: 4 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793336399, i: 4 })
```

Habilitamos el sharding para nuestra base de datos

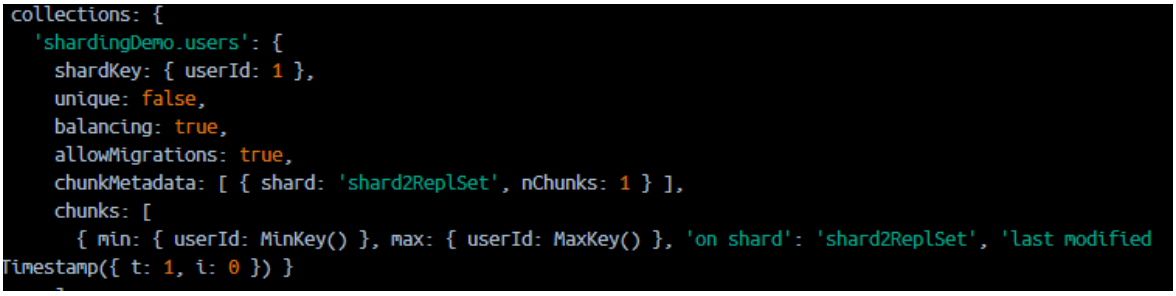
```
[direct: mongos] test> sh.enableSharding("shardingDemo")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793336613, i: 8 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793336613, i: 2 })
}
[direct: mongos] test>
```

Le informamos a la base que estamos repartiendo los datos

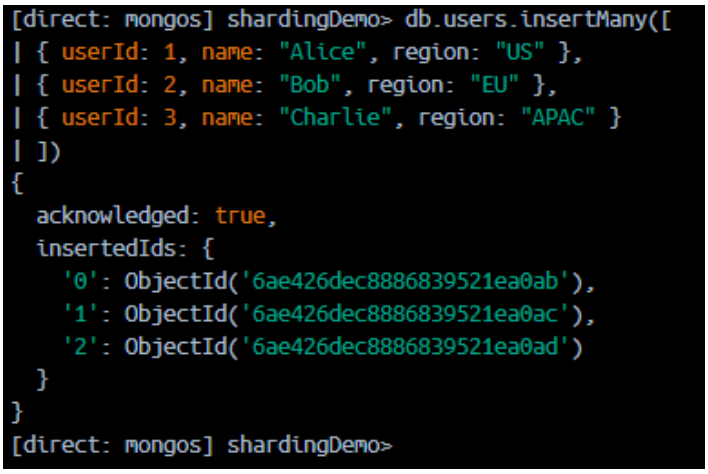
```
[direct: mongos] test> sh.shardCollection("shardingDemo.users", { userId: 1 })
{
  collectionsharded: 'shardingDemo.users',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793336711, i: 36 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793336711, i: 36 })
}
[direct: mongos] test>
```

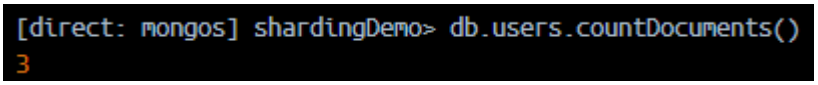
Podemos ver como nuestra base de datos “sharding demo” ya está creada

```
database: {
  _id: 'shardingDemo',
  primary: 'shard2ReplSet',
  partitioned: false,
  version: {
    uuid: UUID('0ef93e30-301d-49ee-9381-64b5bcc41839'),
    timestamp: Timestamp({ t: 1793336612, i: 1 }),
    lastMod: 1
  }
},
```

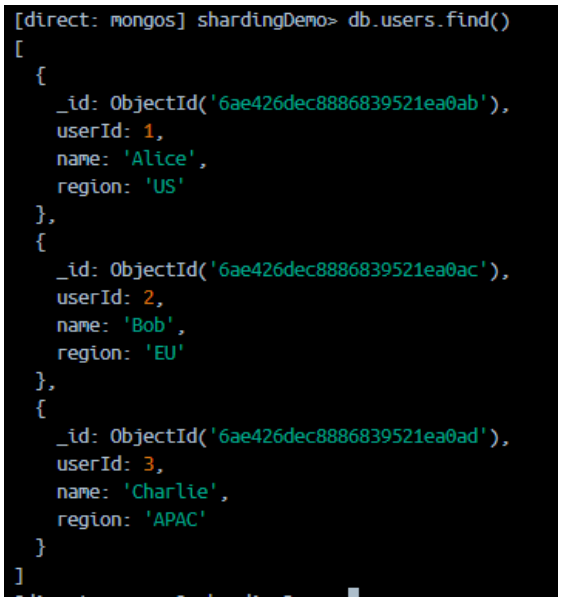


Ahora le insertamos datos a la base





Leemos los datos



Añadimos 10000 registros para probar el auto splitting

```
[direct: mongos] shardingDemo> for (let i = 4; i <= 10000; i++) {
| db.users.insertOne({ userId: i, name: "User" + i, region: "Region" + (i % 5) });
| }
{
  acknowledged: true,
  insertedId: ObjectId('6ae42855c8886839521ec7ba')
}
[direct: mongos] shardingDemo>
```

```
[direct: mongos] shardingDemo> db.users.countDocuments()
10000
```

Vemos la distribución de los datos

```
[direct: mongos] shardingDemo> db.users.getShardDistribution()
Shard shard2ReplSet at shard2ReplSet/shard2:27019
{
  data: '711KiB',
  docs: 10000,
  chunks: 1,
  'estimated data per chunk': '711KiB',
  'estimated docs per chunk': 10000
}
---
Totals
{
  data: '711KiB',
  docs: 10000,
  chunks: 1,
  'Shard shard2ReplSet': [
    '100 % data',
    '100 % docs in cluster',
    '72B avg obj size on shard'
  ]
}
[direct: mongos] shardingDemo>
```

Solo el shard 2 fué usado, esto se debe a que nuestro volumen de datos es demasiado pequeño (aproximadamente 70 bytes), el volumen necesario para iniciar una distribución automática de los datos sería de 64 megabytes (aproximadamente 1 millón de registros).

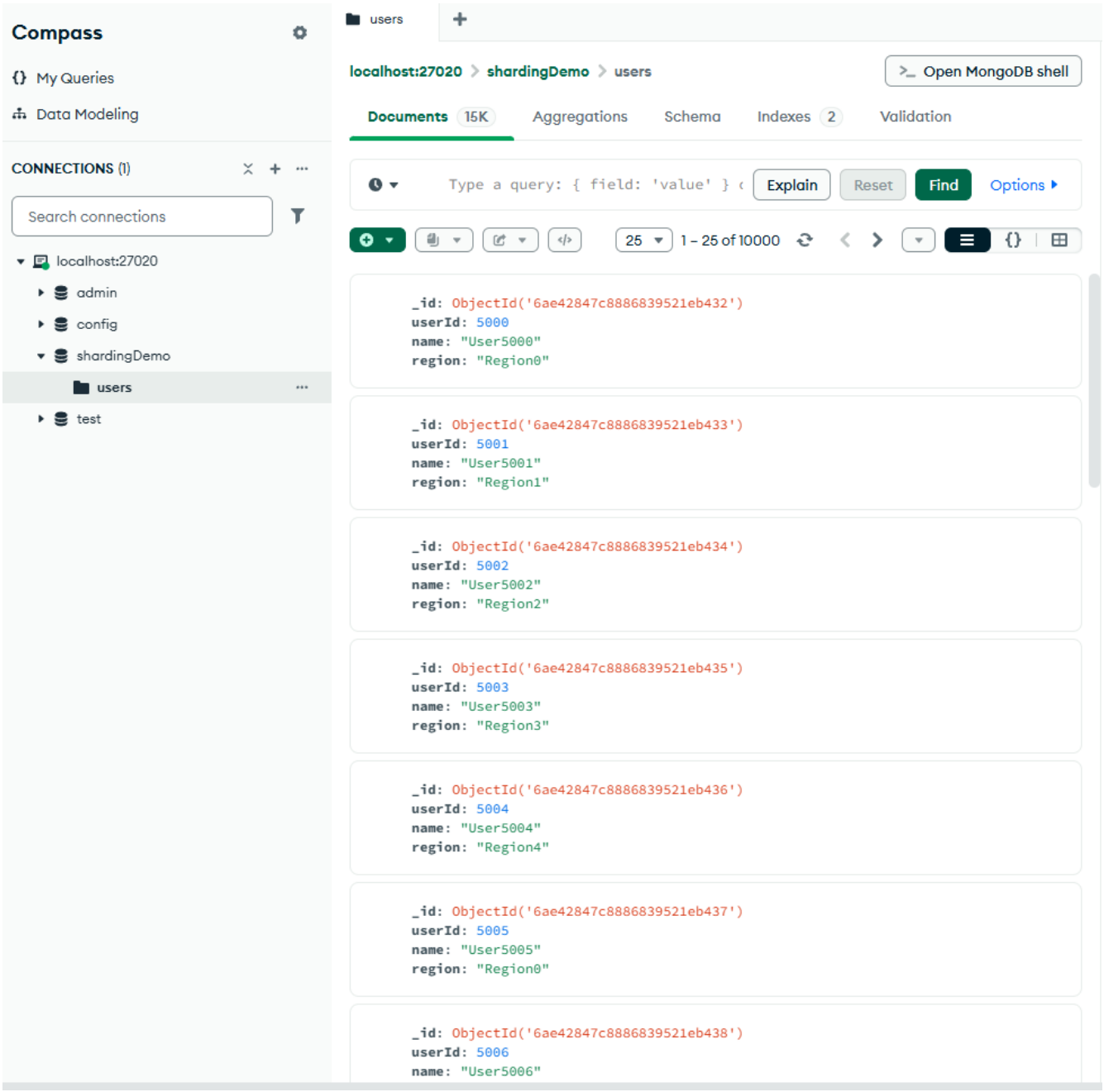
Podemos particionar los datos de manera manual

```
[direct: mongos] shardingDemo> sh.splitAt("shardingDemo.users", { userId: 5000 })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793337944, i: 11 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793337944, i: 11 })
}
[direct: mongos] shardingDemo>
```

```
[direct: mongos] shardingDemo> sh.moveChunk("shardingDemo.users", { userId: 5000 }, "shard1ReplSet")
{
  millis: 262,
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793338065, i: 21 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793338065, i: 21 })
}
```

```
[direct: mongos] shardingDemo> db.users.getShardDistribution()
Shard shard1ReplSet at shard1ReplSet/shard1:27018
{
  data: '356KiB',
  docs: 5001,
  chunks: 1,
  'estimated data per chunk': '356KiB',
  'estimated docs per chunk': 5001
}
---
Shard shard2ReplSet at shard2ReplSet/shard2:27019
{
  data: '360KiB',
  docs: 10000,
  chunks: 1,
  'estimated data per chunk': '360KiB',
  'estimated docs per chunk': 10000
}
---
Totals
{
  data: '716KiB',
  docs: 15001,
  chunks: 2,
  'Shard shard1ReplSet': [
    '49.74 % data',
    '33.33 % docs in cluster',
    '73B avg obj size on shard'
  ],
  'Shard shard2ReplSet': [
    '50.25 % data',
    '66.66 % docs in cluster',
    '72B avg obj size on shard'
  ]
}
[direct: mongos] shardingDemo>
```

Podemos ver como el shard 1 que inicialmente estaba vacío ahora tiene datos



Hacemos operaciones de actualización y eliminación para probar

Delete:

```
[direct: mongos] shardingDemo> db.users.find({region:"US"});
[
  {
    _id: ObjectId('6ae426dec8886839521ea0ab'),
    userId: 1,
    name: 'Alice',
    region: 'US'
  }
]
[direct: mongos] shardingDemo>
```

```
[direct: mongos] shardingDemo> db.users.deleteOne({userId:1});
{ acknowledged: true, deletedCount: 1 }
[direct: mongos] shardingDemo> db.users.find({region:"US"});

[direct: mongos] shardingDemo>
```

Update:

```
[direct: mongos] shardingDemo> db.users.find({region:"Region1"});
[
  {
    _id: ObjectId('6ae42838c8886839521ea0b0'),
    userId: 6,
    name: 'User6',
    region: 'Region1'
  },
  {
    _id: ObjectId('6ae42838c8886839521ea0b5'),
    userId: 11,
    name: 'User11',
    region: 'Region1'
  },
  {
    _id: ObjectId('6ae42839c8886839521ea0ba'),
    userId: 16,
    name: 'User16',
    region: 'Region1'
  },
  {
    _id: ObjectId('6ae42839c8886839521ea0bf'),
    userId: 21,
    name: 'User21',
    region: 'Region1'
  },
```

```
[direct: mongos] shardingDemo> db.users.updateMany({region:"Region1"}, {$set: {region:"Region5"}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1999,
  modifiedCount: 1999,
  upsertedCount: 0
}
[direct: mongos] shardingDemo> db.users.find({region:"Region1"});

[direct: mongos] shardingDemo>
```

```
[direct: mongos] shardingDemo> db.users.find({region:"Region5"});
[
  {
    _id: ObjectId('6ae42838c8886839521ea0b0'),
    userId: 6,
    name: 'User6',
    region: 'Region5'
  },
  {
    _id: ObjectId('6ae42838c8886839521ea0b5'),
    userId: 11,
    name: 'User11',
    region: 'Region5'
  },
  {
    _id: ObjectId('6ae42839c8886839521ea0ba'),
    userId: 16,
    name: 'User16',
    region: 'Region5'
  },
  {
    _id: ObjectId('6ae42839c8886839521ea0bf'),
    userId: 21,
    name: 'User21',
    region: 'Region5'
  },
```